LEARNING GUIDE · MINI PROJECT AI

Knight Trap

Implementasi AI pada Game Knight Trap Menggunakan Algoritma Minimax dengan Alpha-Beta Pruning

Clarrence Adriano Hemeldan 0706012410066

Bagianmu: Antarmuka (UI), BFS & wilayah (territory), uji win rate dan waktu respons

Artificial Intelligence · Information Management & Technology · Universitas Ciputra 2026/2027

Disusun dari proposal (v1.1) dan kode di repo KnightTrap · 29 Oktober 2026

Daftar Isi

1. Peta Belajarmu
2. Gambaran Proyek (Proposal Bab 1)
3. Aturan Game → Kode (Proposal Bab 2.1)
4. Alur Satu Giliran di Kode
5. Konsep AI yang Wajib Dipahami Semua Anggota
6. Antarmuka: Bagaimana Papan Digambar
7. BFS: Breadth-First Search
8. Wilayah (Territory) dan Skor Evaluasi
9. Pemain Pembanding (Baseline)
10. Pengujian Win Rate dan Waktu Respons
11. Hasil Pengujian (Proposal Bab 2.2)
12. Fitur Rally: Poin, Leaderboard, Admin
13. Latihan
14. Pertanyaan Presentasi yang Mungkin Muncul
15. Checklist Sebelum Presentasi

1 Peta Belajarmu

Setelah mempelajari guide ini, kamu bisa:

- Menjelaskan bagaimana papan, knight, kotak terbakar, jebakan, dan penanda lompatan legal digambar.
- Menjelaskan BFS langkah demi langkah dan menghitung jarak knight dengan tangan.
- Menjelaskan cara wilayah (territory) dihitung dari dua hasil BFS, termasuk aturan seri.
- Menjelaskan pemain pembanding (acak dan greedy) serta cara win rate dan waktu respons diukur.

Tugasmu di Timeline (Bab 2.3)

Minggu	Pekerjaan	Kode
1–2	Antarmuka: papan, knight, kotak terbakar/jebakan, penanda lompatan legal, penunjuk giliran, pesan menang/kalah	`board.js`, `fx.js`, `css/style.css`, `index.html`, `main.js` (`ui`)
2–3	BFS: jumlah lompatan ke setiap kotak, wilayah, skor evaluasi	`ai.js`: `bfs()`, bagian wilayah di `evaluate()`
3	Penyatuan BFS + DFS ke evaluasi (bersama Shatrya)	`ai.js`: `evaluate()`
3	Simulasi win rate vs acak & greedy, waktu respons per level	`tests/run.mjs` bagian 3–4; `randomMove()`, `greedyMove()`

2 Gambaran Proyek (Proposal Bab 1)

Apa yang kita buat

Knight Trap adalah varian dari permainan papan *Isolation* (1972). Dua knight ada di papan 7 × 7. Pada gilirannya, pemain:

1. **Lompat**: menggerakkan knight seperti kuda catur (huruf L) ke kotak terbuka.
2. **Bakar**: kotak yang ditinggalkan langsung terbakar dan tidak bisa dipakai lagi.
3. **Jebak**: menghancurkan satu kotak terbuka lain di mana pun.

Pemain yang **tidak punya lompatan legal** pada gilirannya kalah. Game ini dibuat untuk **rally games** (NPLC Universitas Ciputra, tema *Human vs AI*), jadi aturannya harus bisa dijelaskan dalam satu menit, satu game selesai dalam beberapa menit, dan lawan AI-nya tidak boleh terlalu mudah.

Masalah yang diselesaikan (Bab 1.2)

Kita harus membuat lawan AI yang (1) memilih langkah yang kuat, (2) cukup cepat untuk browser, dan (3) punya beberapa tingkat kesulitan. Ada dua tantangan utama:

- **Pohon permainan besar.** Satu langkah = pilihan lompatan (maks. 8) × pilihan kotak jebakan (bisa puluhan). Jumlah posisi meledak setiap giliran.
- **Menilai posisi tanpa main sampai habis.** Kita butuh *fungsi evaluasi* yang memberi skor pada posisi di tengah permainan.

Solusinya, sesuai ketentuan tugas yang membatasi kita ke **empat algoritma**: **Minimax** + **Alpha-Beta Pruning** untuk mengambil keputusan, dan **BFS** + **DFS** sebagai dasar fungsi evaluasi.

Peta file kode

File	Isi	Pemilik utama (Timeline)
`js/config.js`	Konstanta: ukuran papan, posisi awal, kedalaman tiap level, batas AI, poin	Shatrya (setup), Louis (AI)
`js/rules.js`	Aturan game: lompatan legal, bakar, jebakan, cek kalah. Tidak menyentuh DOM	Shatrya
`js/ai.js`	Minimax + Alpha-Beta, BFS, DFS, fungsi evaluasi, pemain acak & greedy	Louis (search), Clarrence (BFS), Shatrya (DFS)
`js/ai.worker.js`, `js/ai-client.js`	Menjalankan AI di Web Worker supaya tampilan tidak macet	Louis
`js/game.js`	Alur giliran: input pemain, giliran AI, animasi, game over	Shatrya & Louis
`js/board.js`, `js/fx.js`, `css/style.css`, `index.html`	Tampilan papan, knight, efek api/jebakan, layout	Clarrence
`js/main.js`, `js/i18n.js`, `js/audio.js`	Layar menu, pengaturan, panel statistik, teks dua bahasa, suara	Clarrence & Louis
`js/net.js`	Multiplayer online (PeerJS)	Tambahan di luar proposal
`js/scoring.js`, `api/*`, `admin.html`	Poin, leaderboard, halaman admin (fitur rally)	Tambahan di luar proposal
`tests/run.mjs`	Semua pengujian Bab 2.2	Louis, Clarrence (pengujian)

> **Repo dengan komentar per baris.** Folder `KnightTrap-Annotated` berisi kode yang sama persis, tetapi setiap baris diberi komentar bahasa Indonesia. Pakai repo itu untuk belajar; potongan kode di guide ini diambil dari repo asli `KnightTrap` supaya nomor barisnya cocok dengan kode yang dijalankan.

3 Aturan Game → Kode (Proposal Bab 2.1)

Setiap aturan di proposal punya tempat yang jelas di `js/rules.js`. File ini adalah **satu-satunya sumber aturan**: tampilan, AI, mode online, dan server leaderboard semuanya memakai fungsi yang sama.

Aturan proposal	Kode
Papan 7 × 7, array 49 kotak (kosong / terbakar / jebakan) + posisi 2 knight	`createState()`, konstanta `EMPTY`, `BURNED`, `TRAP`
(a) Knight bergerak huruf L	`JUMPS` dan `NEIGHBORS` (dihitung sekali di awal)
(b) Kotak yang ditinggalkan terbakar	`applyJump()`
(c) Hancurkan satu kotak terbuka	`applyTrap()`; validasi di `validateMove()`
(d) Tidak boleh mendarat di kotak terbakar, jebakan, atau kotak lawan	`isOpen()` dipakai oleh `legalJumps()`
(e) Tidak punya lompatan legal = kalah	`winner()` memakai `canMove()`

Representasi papan

Kotak diberi nomor 0 sampai 48: `index = baris × 7 + kolom`, baris 0 adalah baris paling atas (rank 7). Jadi **a1 = 42** (kiri bawah) dan **g7 = 6** (kanan atas).

js/config.js : baris 3–8

```
3  export const N = 7;
4  export const SIZE = N * N;
5
6  // The proposal does not fix start squares. P1 starts at a1 (bottom-left),
7  // P2 at g7 (top-right). Index = row * 7 + col, row 0 is the top rank (7).
8  export const START = Object.freeze({ p1: 42, p2: 6, first: 0 });
```

js/rules.js : baris 28–35

```
28 export function createState(start = START) {
29   return {
30     cells: new Int8Array(SIZE),
31     pos: [start.p1, start.p2],
32     turn: start.first ?? 0,
33     moves: 0,
34   };
35 }
```

`cells` memakai `Int8Array` (array angka 8-bit) karena kecil dan cepat disalin. `pos[0]` adalah posisi knight P1, `pos[1]` knight P2, `turn` adalah siapa yang jalan (0 atau 1).

Tetangga knight dihitung sekali

js/rules.js : baris 10–26

```
const JUMPS = [
  [-2, -1], [-2, 1], [-1, -2], [-1, 2],
  [1, -2], [1, 2], [2, -1], [2, 1],
];

// Knight neighbor lists, computed once.
export const NEIGHBORS = Array.from({ length: SIZE }, (_, i) => {
  const r = Math.floor(i / N);
  const c = i % N;
  const out = [];
  for (const [dr, dc] of JUMPS) {
    const rr = r + dr;
    const cc = c + dc;
    if (rr >= 0 && rr < N && cc >= 0 && cc < N) out.push(rr * N + cc);
  }
  return out;
});
```

Untuk setiap kotak, kita coba 8 arah L dan simpan hanya yang masih di dalam papan. Hasilnya tabel 49 daftar. AI memanggil daftar ini jutaan kali, jadi menghitungnya sekali di awal menghemat banyak waktu.

Satu giliran = lompat, bakar, jebak

js/rules.js : baris 53–55

```
export function isOpen(s, sq) {
  return s.cells[sq] === EMPTY && sq !== s.pos[0] && sq !== s.pos[1];
}
```

js/rules.js : baris 57–61

```
export function legalJumps(s, side = s.turn) {
  const out = [];
  for (const n of NEIGHBORS[s.pos[side]]) if (isOpen(s, n)) out.push(n);
  return out;
}
```

js/rules.js : baris 74–80

```
export function applyJump(s, to) {
  const side = s.turn;
  const from = s.pos[side];
  s.cells[from] = BURNED;
  s.pos[side] = to;
  return from;
}
```

js/rules.js : baris 83–85

```
export function applyTrap(s, sq) {
  if (sq >= 0) s.cells[sq] = TRAP;
}
```

js/rules.js : baris 111–113

```
111  export function winner(s) {
112    return canMove(s, s.turn) ? null : s.turn ^ 1;
113  }
```

Kenapa `winner()` mengembalikan `s.turn ^ 1`? `^` adalah XOR. `0 ^ 1 = 1` dan `1 ^ 1 = 0`, jadi ini cara singkat menulis "pemain lainnya". Kalau yang sedang jalan tidak bisa bergerak, lawannya menang.

4 Alur Satu Giliran di Kode

Bagian ini menghubungkan semua file. Setiap anggota harus bisa menceritakan alur ini saat presentasi.



Gambar: alur satu ronde (pemain lalu AI). Biru = input pemain, krem = alur game, oranye = AI.

js/game.js : baris 119–136

```
119   nextTurn() {
120     if (this.disposed) return;
121     const w = winner(this.state);
122     if (w !== null) return this.finish(w);
123     const p = this.current;
124     if (p.kind === 'human') {
125       this.phase = 'jump';
126       if (this.mode === 'local' && this.history.length) sfx.turn();
127       this.refresh();
128     } else if (p.kind === 'ai') {
129       this.phase = 'ai';
130       this.refresh();
131       this.runAi();
132     } else {
133       this.phase = 'remote';
134       this.refresh();
135     }
136   }
```

js/game.js : baris 138–162

```
  async runAi() {
    this.ui.thinking(true);
    const t0 = performance.now();
    let res = null;
    try {
      res = await think(this.state, { depth: this.ai.depth, prune: this.ai.prune });
    } catch (err) {
      console.error('AI failed', err);
    }
    if (this.disposed) return;
    const elapsed = performance.now() - t0;
    if (elapsed < AICFG.minUiDelayMs) await sleep(AICFG.minUiDelayMs - elapsed);
    if (this.disposed) return;
    this.ui.thinking(false);
    let move = res?.move;
    if (!move || !validateMove(this.state, move)) {
      console.warn('AI returned no valid move; falling back to a random legal move');
      move = randomMove(this.state);
    } else {
      this.ui.stats(res);
    }
    await this.playMove(move);
    if (this.disposed) return;
    this.nextTurn();
  }
```

Perhatikan dua pengaman di `runAi`: (1) kalau AI terlalu cepat (Easy), tampilan menunggu minimal 450 ms supaya tidak terasa "langsung"; (2) kalau AI gagal atau mengembalikan langkah tidak sah, game memakai langkah acak yang legal, jadi game tidak pernah macet.

js/ai-client.js : baris 44–53

```
export function think(state, { depth, prune }) {
  const args = { state: serialize(state), depth, prune };
  const w = getWorker();
  if (!w) return runInline(args);
  return new Promise((resolve, reject) => {
    const id = ++seq;
    pending.set(id, { resolve, reject, args });
    w.postMessage({ id, ...args });
  });
}
```

js/ai.worker.js : baris 5–13

```
self.onmessage = (e) => {
  const { id, state, depth, prune } = e.data;
  try {
    const result = search(deserialize(state), { depth, prune });
    self.postMessage({ id, result });
  } catch (err) {
    self.postMessage({ id, error: String(err && err.message ? err.message : err) });
  }
};
```

Kenapa Web Worker? JavaScript di browser berjalan di satu thread. Kalau Expert berpikir 1–2 detik di thread utama, animasi dan tombol akan membeku. Worker menjalankan `search()` di thread terpisah, jadi papan tetap hidup saat AI berpikir.

5 Konsep AI yang Wajib Dipahami Semua Anggota

Dosen bisa bertanya ke siapa saja. Bagian ini adalah dasar yang harus dikuasai semua anggota, walaupun detailnya milik anggota lain.

Agen dan model P.E.A.S (Tabel 2.1.1)

Komponen	Di Knight Trap	Di kode
Performance	Win rate AI, waktu berpikir ≤ 4 detik, jumlah posisi diperiksa sedikit, tantangan pas untuk siswa SMA	`tests/run.mjs`, panel statistik
Environment	Papan 7 × 7, dua agen bergiliran. Deterministik, fully observable, diskret, berurutan, zero-sum	`createState()`
Actuators	Lompat ke kotak legal (maks. 8 arah) + hancurkan satu kotak	`applyMove()`, `{ jump, trap }`
Sensors	Posisi kedua knight, status 49 kotak, giliran	`state.cells`, `state.pos`, `state.turn`

Kenapa sifat lingkungan penting? Minimax hanya cocok untuk game *deterministik* (tidak ada dadu), *fully observable* (tidak ada kartu tertutup), dan *zero-sum* (untung saya = rugi lawan). Knight Trap memenuhi ketiganya.

Adversarial search dan game tree

Game tree: akar = posisi sekarang, cabang = langkah, simpul = posisi hasil langkah. **Kedalaman** (depth) = berapa giliran ke depan yang dilihat AI. **Branching factor** = rata-rata jumlah langkah per posisi. Di Knight Trap: sampai 8 lompatan × sampai 12 kandidat jebakan = sampai **96 langkah** per posisi. Dengan depth 4: $96^4 \approx 85$ juta posisi. Itulah alasan kita butuh Alpha-Beta dan pembatasan kandidat.

Minimax dalam satu paragraf

AI adalah **MAX** (ingin skor setinggi mungkin), manusia adalah **MIN** (dianggap selalu memilih langkah terburuk untuk AI). AI menelusuri pohon sampai kedalaman tertentu, menilai posisi di ujung dengan fungsi evaluasi, lalu menaikkan nilai ke atas: di simpul MAX ambil nilai terbesar, di simpul MIN ambil nilai terkecil. Langkah di akar dengan nilai terbesar dipilih.

Alpha-Beta dalam satu paragraf

Alpha = nilai terbaik yang sudah pasti didapat MAX. **Beta** = nilai terbaik yang sudah pasti didapat MIN. Kalau di suatu simpul `beta ≤ alpha`, sisa langkah di simpul itu tidak mungkin mengubah keputusan, jadi dilewati (**cutoff**). Hasilnya **sama persis** dengan Minimax biasa, tapi posisi yang diperiksa jauh lebih sedikit. Ini dibuktikan di pengujian.

BFS dan DFS dalam satu paragraf

BFS (antrian, menyebar per "lapisan") menghitung jumlah lompatan minimum dari knight ke setiap kotak. Kotak yang lebih cepat dicapai knight A adalah **wilayah** A. **DFS** (tumpukan, masuk sedalam mungkin dulu)

melakukan **flood fill**: menandai semua kotak yang bisa dicapai knight. Kalau area kedua knight tidak bersentuhan, papan sudah **terbelah** dan pemenangnya hampir pasti yang areanya lebih luas.

js/ai.js : baris 78–108

```
78  export function evaluate(s, me) {
79    const { cells, pos, turn } = s;
80    const areaA = dfs(cells, pos, 0, markA);
81    const areaB = dfs(cells, pos, 1, markB);
82    let split = true;
83    for (let i = 0; i < SIZE; i++) {
84      if (markA[i] === 1 && markB[i] === 1) {
85        split = false;
86        break;
87      }
88    }
89    let score;
90    if (split) {
91      score = (areaA - areaB) * SPLIT_WEIGHT;
92    } else {
93      bfs(cells, pos, 0, distA);
94      bfs(cells, pos, 1, distB);
95      let terr = 0;
96      for (let i = 0; i < SIZE; i++) {
97        if (!open(cells, pos, i)) continue;
98        const a = distA[i];
99        const b = distB[i];
100       if (a < 0 && b < 0) continue;
101       if (b < 0 || (a >= 0 && a < b)) terr++;
102       else if (a < 0 || b < a) terr--;
103       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
104     }
105     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
106   }
107   return me === 0 ? score : -score;
108 }
```

Inti fungsi evaluasi (skor dari sudut pandang P1, lalu dibalik tandanya untuk P2):

- Kalau papan sudah terbelah (DFS): `skor = (areaP1 − areaP2) × 100`.
- Kalau belum: `skor = selisih wilayah (BFS) + 0,5 × selisih mobilitas`. Mobilitas = jumlah lompatan legal saat ini.

Angka 100 (`SPLIT_WEIGHT`) dipilih karena skor wilayah paling besar sekitar 53, jadi papan yang sudah terbelah selalu dinilai lebih "pasti" daripada posisi yang masih terbuka.

6 Antarmuka: Bagaimana Papan Digambar

Prinsip: tampilan tidak menyimpan aturan

`board.js` hanya **menggambar**. Ia tidak tahu aturan dan tidak menyimpan state game. `game.js` memberi tahu apa yang harus digambar lewat objek `view`. Pemisahan ini membuat aturan cukup ditulis sekali di `rules.js`.

js/board.js : baris 15–57

```
15    constructor(boardEl, shellEl) {
16      this.el = boardEl;
17      this.shell = shellEl;
18      this.onCell = null;
19      this.onKnight = null;
20      this.cells = [];
21
22      const frag = document.createDocumentFragment();
23      for (let sq = 0; sq < SIZE; sq++) {
24        const { r, c } = rc(sq);
25        const b = document.createElement('button');
26        b.type = 'button';
27        b.className = 'cell';
28        b.dataset.sq = sq;
29        if ((r + c) % 2 === 1) b.classList.add('dark');
30        if (r === N - 1) b.insertAdjacentHTML('beforeend', `<span class="coord f">${'abcdefg'[c]}
</span>`);
31        if (c === 0) b.insertAdjacentHTML('beforeend', `<span class="coord r">${N - r}</span>`);
32        frag.appendChild(b);
33        this.cells.push(b);
34      }
35      this.el.appendChild(frag);
36
37      this.knights = [0, 1].map((side) => {
38        const k = document.createElement('div');
39        k.className = 'knight';
40        k.dataset.side = side;
41        k.innerHTML = knightSvg(side);
42        k.addEventListener('click', () => this.onKnight?.(side));
43        this.el.appendChild(k);
44        return k;
45      });
46
47      const canvas = document.createElement('canvas');
48      canvas.className = 'fx';
49      this.el.appendChild(canvas);
50      this.fxCanvas = canvas;
51      this.particles = new Particles(canvas);
52
53      this.el.addEventListener('click', (e) => {
54        const b = e.target.closest('.cell');
55        if (b) this.onCell?.(Number(b.dataset.sq));
56      });
57    }
```

- 49 `<button>` dibuat sekali, bukan setiap giliran. Tombol dipakai (bukan `div`) supaya bisa diklik dengan keyboard dan dibaca screen reader.
- Kotak gelap/terang memakai rumus `(r + c) % 2`, sama seperti papan catur.

- Label koordinat (a–g, 1–7) hanya di baris bawah dan kolom kiri.
- Satu listener klik di papan (*event delegation*), bukan 49 listener. `closest('.cell')` mencari kotak yang diklik.
- Kanvas `.fx` di atas papan untuk partikel api dan debu.

js/board.js : baris 69–102

```
69    render(view) {
70      const { state } = view;
71      const legal = new Set(view.legal || []);
72      const threats = new Set(view.threats || []);
73      for (let sq = 0; sq < SIZE; sq++) {
74        const b = this.cells[sq];
75        const v = state.cells[sq];
76        const occupied = sq === state.pos[0] || sq === state.pos[1];
77        b.classList.toggle('burned', v === BURNED);
78        b.classList.toggle('trap', v === TRAP);
79        const isLegal = legal.has(sq);
80        const isTrappable = view.trappable && v === 0 && !occupied;
81        b.classList.toggle('legal', isLegal);
82        b.classList.toggle('trappable', isTrappable);
83        b.classList.toggle('threat', isTrappable && threats.has(sq));
84        b.classList.toggle('last-land', sq === view.lastLand);
85        b.classList.toggle('last-trap', sq === view.lastTrap && v === TRAP);
86        const actionable = isLegal || isTrappable;
87        b.tabIndex = actionable ? 0 : -1;
88        b.setAttribute('aria-disabled', actionable ? 'false' : 'true');
89        const what = v === BURNED ? 'burned' : v === TRAP ? 'trap' : occupied ? 'knight' : 'open';
90        b.setAttribute('aria-label', `${squareName(sq)} ${what}${isLegal ? ', jump here' :
''}${isTrappable ? ', place trap' : ''}`);
91      }
92      this.el.dataset.side = view.turnSide ?? '';
93      this.shell.dataset.turn = view.turnSide ?? '';
94      this.shell.classList.toggle('trap-mode', !!view.trappable);
95      for (const side of [0, 1]) {
96        const k = this.knights[side];
97        this.placeKnight(side, state.pos[side]);
98        k.classList.toggle('is-turn', view.turnSide === side && view.dead == null);
99        k.classList.toggle('is-dead', view.dead === side);
100       k.classList.toggle('selectable', view.selectableKnight === side);
101     }
102   }
```

`render()` dipanggil setiap ada perubahan. Ia hanya menyalakan/mematikan **class CSS**: `burned`, `trap`, `legal` (titik penanda lompatan), `trappable`, `threat` (kotak tujuan lawan yang bagus untuk dijebak), `last-land`, `last-trap`. Semua tampilan visual ditentukan CSS. `aria-label` diisi supaya pembaca layar bisa menyebut "d4 open, jump here".

css/style.css : baris 638–665

```
.board {
  --gap: 3px;
  position: relative;
  width: 100%;
  height: 100%;
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  grid-template-rows: repeat(7, 1fr);
  gap: var(--gap);
  border-radius: 14px;
  overflow: visible;
}

.cell {
  position: relative;
  border: 0;
  padding: 0;
  border-radius: 8px;
  background:
    linear-gradient(180deg, rgba(255, 255, 255, 0.05), rgba(0, 0, 0, 0.12)),
    var(--cell-a);
  box-shadow: 0 1px 0 rgba(255, 255, 255, 0.05) inset, 0 -1px 0 rgba(0, 0, 0, 0.35) inset;
  cursor: default;
  transition: background 0.35s, transform 0.2s var(--ease-out), box-shadow 0.3s;
  overflow: hidden;
}
.cell.dark { background: linear-gradient(180deg, rgba(255, 255, 255, 0.035), rgba(0, 0, 0, 0.14)),
var(--cell-b); }
.cell .coord {
```

Papan memakai **CSS Grid** 7 × 7 (`repeat(7, 1fr)`), jadi ukurannya otomatis mengikuti layar. Warna diambil dari variabel di `:root` (misalnya `--cell-a`), sehingga tema cukup diubah di satu tempat.

Animasi lompatan

js/board.js : baris 104–131

```
104   async animateJump(side, from, to) {
105     const k = this.knights[side];
106     this.placeKnight(side, to);
107     if (reducedMotion()) return;
108     const a = rc(from);
109     const b = rc(to);
110     const mc = (a.c + b.c) / 2;
111     const mr = (a.r + b.r) / 2;
112     // Keyframes need a concrete gap; it changes with the breakpoint (--gap in CSS).
113     const gap = parseFloat(getComputedStyle(this.el).columnGap) || 0;
114     const anim = k.animate(
115       [
116         { transform: tf(a.c, a.r, gap, 0, 1), offset: 0 },
117         { transform: tf(mc, mr, gap, -42, 1.28), offset: 0.5 },
118         { transform: tf(b.c, b.r, gap, 4, 0.94), offset: 0.88 },
119         { transform: tf(b.c, b.r, gap, 0, 1), offset: 1 },
120       ],
121       { duration: 480, easing: 'cubic-bezier(.45,.05,.35,1)' },
122     );
123     k.style.zIndex = 5;
124     try {
125       await anim.finished;
126     } catch {
127       /* cancelled */
128     }
129     k.style.zIndex = '';
130     this.fx(to, 'land');
131   }
```

Knight bergerak dalam busur: dari kotak asal, naik ke titik tengah (`translateY(-42%)`, diperbesar 1,28×), turun sedikit melewati tujuan (0,94×), lalu diam. Memakai Web Animations API (`element.animate`). Kalau pengguna memilih *reduce motion* di sistem operasinya, animasi dilewati. Setelah mendarat, efek debu `'land'` dipanggil.

js/board.js : baris 157–160

```
157   burn(sq) {
158     this.pulse(sq, 'ignite');
159     this.fx(sq, 'ember');
160   }
```

js/board.js : baris 162–172

```
162   async trap(sq) {
163     this.pulse(sq, 'drop');
164     this.fx(sq, 'debris');
165     if (!reducedMotion()) {
166       this.shell.classList.remove('shake');
167       void this.shell.offsetWidth;
168       this.shell.classList.add('shake');
169       setTimeout(() => this.shell.classList.remove('shake'), 350);
170     }
171     await sleep(reducedMotion() ? 0 : 260);
172   }
```

Penunjuk giliran dan pesan menang/kalah

js/main.js : baris 369–388

```
369    update(g, text) {
370      const s = g.state;
371      const status = $('#status');
372      const side = g.phase === 'over' ? g.winner : s.turn;
373      status.dataset.side = side;
374      status.classList.toggle('trap-mode', g.phase === 'trap');
375      $('#status-text').textContent = text;
376      $('#undo-jump').hidden = !(g.phase === 'trap' && g.current?.kind === 'human');
377      for (const card of $$('.pcard')) {
378        const sd = Number(card.dataset.side);
379        const p = g.players[sd];
380        $('.pcard-name', card).textContent = p.name;
381        $('.pcard-sub', card).textContent = p.sub;
382        const mob = legalJumps(s, sd).length;
383        $('.pcard-mob b', card).textContent = mob;
384        $('.pcard-mob', card).classList.toggle('danger', g.phase !== 'over' && mob <= 1);
385        card.classList.toggle('is-turn', g.phase !== 'over' && s.turn === sd);
386        card.classList.toggle('is-winner', g.phase === 'over' && g.winner === sd);
387      }
388    },
```

Setiap refresh, kartu pemain menampilkan nama, warna, dan **mobilitas** (jumlah lompatan legal). Kalau tinggal ≤ 1, angkanya merah berkedip (`danger`), jadi pemain tahu sedang terancam. Status di atas papan berubah warna sesuai giliran, dan menjadi merah saat fase jebakan.

js/main.js : baris 431–455

```
431    over(info) {
432      const g = info.game;
433      const me = g.mySide;
434      const winName = g.players[info.winner].name;
435      const loseName = g.players[info.loser].name;
436      let title;
437      let sub;
438      let win = true;
439      if (g.mode === 'local') {
440        title = t('sideWins', winName);
441        sub = t('trappedSub', loseName);
442      } else {
443        win = me === info.winner;
444        title = win ? t('youWin') : t('youLose');
445        if (g.mode === 'ai') sub = win ? t('youWinSub') : t('aiWinSub');
446        else sub = t('trappedSub', loseName);
447      }
448      let score = null;
449      if (pointsEnabled(g.mode)) {
450        score = currentScore(g);
451        renderBreakdown(score, g.level, g.rules);
452      }
453      openGameOver({ title, sub, win, winnerSide: info.winner, moves: info.moves, score });
454      if (score) lastGame = { difficulty: g.level, side: me, id: newGameId(), moves: gameMoves(g), sent:
false };
455    },
```

Efek partikel

`fx.js` menggambar api (`ember`), debu (`debris`), debu mendarat (`land`), kembang api menang (`victory`), dan label poin "+N" di sebuah `<canvas>`. Setiap partikel punya posisi, kecepatan, gravitasi, umur; setiap frame (`requestAnimationFrame`) posisinya diperbarui dan partikel yang habis umurnya dibuang. Ini hanya tampilan, tidak mempengaruhi AI.

7 BFS: Breadth-First Search

Ide

BFS menelusuri graf **per lapisan**: semua kotak berjarak 1 lompatan, lalu semua kotak berjarak 2, dan seterusnya. Karena itu, saat sebuah kotak pertama kali ditemukan, jaraknya pasti yang **terpendek**. Struktur datanya adalah **antrian** (queue, FIFO): yang masuk pertama keluar pertama.

Di sini, **graf** = 49 kotak, dan dua kotak terhubung kalau bisa dicapai dengan satu lompatan knight dan kotak tujuannya masih terbuka.

7	4	3	4	3	4	5	4
6	3	4	3	4	3	4	5
5	2	3	2	3	4	3	4
4	3	2	3	2	3	4	3
3	2	1	4	3	2	3	4
2	3	4	1	2	3	4	3
1	♘	3	2	3	2	3	4
	a	b	c	d	e	f	g

BFS dari knight di a1: angka = jumlah lompatan minimum ke kotak itu (papan kosong).

7	2	3	2	3	2	3	2
6	3	4	1	2	1	4	3
5	2	1	2	3	2	1	2
4	3	2	3	♘	3	2	3
3	2	1	2	3	2	1	2
2	3	4	1	2	1	4	3
1	2	3	2	3	2	3	2
	a	b	c	d	e	f	g

BFS dari knight di d4: angka = jumlah lompatan minimum ke kotak itu (papan kosong).

Perhatikan: dari pojok a1 butuh sampai 5 lompatan untuk mencapai sebagian kotak, sedangkan dari tengah d4 paling jauh 4. Knight di tengah menguasai papan lebih cepat, dan itulah yang diukur oleh wilayah.

Kode BFS

js/ai.js : baris 36–53

```
36  export function bfs(cells, pos, side, dist) {
37    dist.fill(-1);
38    const start = pos[side];
39    dist[start] = 0;
40    let head = 0;
41    let tail = 0;
42    queue[tail++] = start;
43    while (head < tail) {
44      const cur = queue[head++];
45      const d = dist[cur] + 1;
46      for (const n of NEIGHBORS[cur]) {
47        if (dist[n] < 0 && open(cells, pos, n)) {
48          dist[n] = d;
49          queue[tail++] = n;
50        }
51      }
52    }
53  }
```

Baris	Arti
`dist.fill(-1)`	Semua kotak awalnya "belum dicapai" (−1).
`dist[start] = 0`	Kotak knight sendiri berjarak 0.
`queue`, `head`, `tail`	Antrian dibuat dari array biasa: `head` menunjuk yang akan diambil, `tail` tempat memasukkan. Lebih cepat daripada `array.shift()`.
`while (head < tail)`	Selama antrian belum kosong.
`const d = dist[cur] + 1`	Tetangga kotak ini berjarak satu lebih jauh.
`if (dist[n] < 0 && open(...))`	Hanya kotak yang belum dikunjungi dan masih terbuka (tidak terbakar, tidak dijebak, tidak ditempati knight lawan).

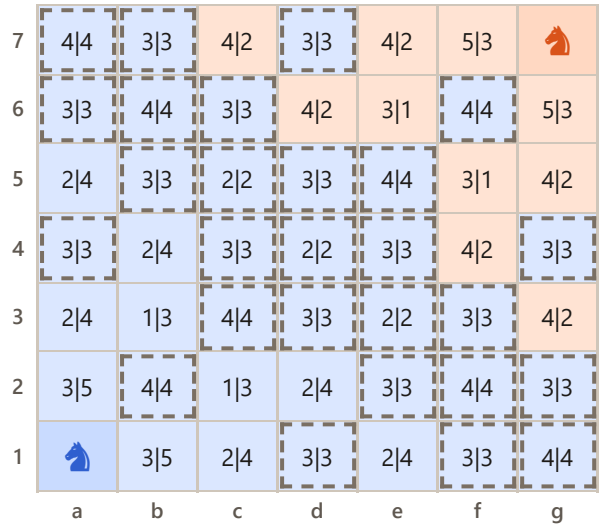
> **Kenapa buffer dibuat sekali di luar fungsi?** `distA`, `distB`, `queue` dibuat sekali sebagai `Int8Array` di atas file. Evaluasi dipanggil puluhan ribu kali per langkah AI; membuat array baru setiap kali akan memperlambat dan membebani garbage collector.

Kompleksitas: setiap kotak masuk antrian paling banyak sekali dan setiap kotak punya maks. 8 tetangga, jadi O(V + E) = maks. 49 + 49 × 8 langkah. Sangat murah.

8 Wilayah (Territory) dan Skor Evaluasi

BFS dijalankan dua kali: dari knight P1 (`distA`) dan dari knight P2 (`distB`). Lalu untuk setiap kotak terbuka:

- Hanya P1 yang bisa mencapai, atau P1 lebih cepat (`a < b`) → wilayah P1 (+1).
- Hanya P2, atau P2 lebih cepat → wilayah P2 (−1).
- **Seri** (`a == b`) → untuk pemain yang jalan berikutnya, karena dia akan sampai duluan.
- Tidak bisa dicapai siapa pun → diabaikan.



Wilayah di posisi awal. Isi kotak = jarak Biru | jarak Oranye. Biru = wilayah P1 (a1), oranye = wilayah P2 (g7), garis putus = seri (dihitung untuk yang jalan berikutnya, P1).

Hasil: P1 37 kotak, P2 10 kotak (termasuk kotak seri untuk P1). Selisih wilayah = 27.

`js/ai.js : baris 92–107`

```
92    } else {
93      bfs(cells, pos, 0, distA);
94      bfs(cells, pos, 1, distB);
95      let terr = 0;
96      for (let i = 0; i < SIZE; i++) {
97        if (!open(cells, pos, i)) continue;
98        const a = distA[i];
99        const b = distB[i];
100       if (a < 0 && b < 0) continue;
101       if (b < 0 || (a >= 0 && a < b)) terr++;
102       else if (a < 0 || b < a) terr--;
103       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
104     }
105     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
106   }
107   return me === 0 ? score : -score;
```

Lalu `skor = terr + 0,5 × (mobilitas P1 − mobilitas P2)`. Mobilitas diberi bobot kecil (0,5) sebagai pemecah seri: dua posisi dengan wilayah sama, yang lompatannya lebih banyak dianggap sedikit lebih baik. Baris terakhir membalik tanda untuk P2, karena skor selalu dari sudut pandang `me` (pemain AI).

> **Kenapa wilayah masuk akal untuk Knight Trap?** Pemain kalah saat kehabisan lompatan. Kotak yang bisa kamu capai lebih dulu adalah kotak yang kemungkinan besar bisa kamu pakai sebelum lawan membakarnya atau menjebaknya. Semakin besar wilayahmu, semakin banyak "nafas" yang tersisa.

Penyatuan dengan DFS: `evaluate()` menjalankan DFS dulu. Kalau papan sudah terbelah, wilayah tidak lagi relevan (setiap knight hanya bisa memakai areanya sendiri), jadi skor dihitung dari luas area × 100. BFS hanya dijalankan kalau papan belum terbelah. Detail DFS ada di guide Shatrya.

9 Pemain Pembanding (Baseline)

Proposal membandingkan AI dengan dua pemain sederhana. Keduanya ada di bagian bawah `js/ai.js`.

js/ai.js : baris 308–316

```
308 export function randomMove(state, rng = Math.random) {
309   const jumps = legalJumps(state);
310   if (jumps.length === 0) return null;
311   const jump = pickRandom(jumps, rng);
312   const t = cloneState(state);
313   applyJump(t, jump);
314   const opens = openSquares(t);
315   return { jump, trap: opens.length ? pickRandom(opens, rng) : -1 };
316 }
```

Pemain acak: pilih lompatan legal secara acak, lalu jebak kotak terbuka secara acak. `rng` bisa diganti generator dengan seed, supaya pengujian bisa diulang.

js/ai.js : baris 320–356

```
320 export function greedyMove(state, rng = Math.random) {
321   const side = state.turn;
322   const jumps = legalJumps(state);
323   if (jumps.length === 0) return null;
324   const bestBy = (list, scoreFn) => {
325     let best = -1;
326     let picks = [];
327     for (const x of list) {
328       const v = scoreFn(x);
329       if (v > best) {
330         best = v;
331         picks = [x];
332       } else if (v === best) picks.push(x);
333     }
334     return pickRandom(picks, rng);
335   };
336   const jump = bestBy(jumps, (j) => {
337     const t = cloneState(state);
338     applyJump(t, j);
339     return legalJumps(t, side).length;
340   });
341   const t = cloneState(state);
342   applyJump(t, jump);
343   if (!hasOpenSquare(t)) return { jump, trap: -1 };
344   const oppJumps = legalJumps(t, side ^ 1);
345   let trap;
346   if (oppJumps.length) {
347     trap = bestBy(oppJumps, (sq) => {
348       let n = 0;
349       for (const k of NEIGHBORS[sq]) if (isOpen(t, k) && k !== sq) n++;
350       return n;
351     });
352   } else {
353     trap = pickRandom(openSquares(t), rng);
354   }
355   return { jump, trap };
356 }
```

Pemain greedy: lompat ke kotak yang memberi lompatan lanjutan terbanyak, lalu jebak kotak tujuan lawan yang paling berharga bagi lawan. Kalau beberapa pilihan sama bagusnya, pilih acak di antaranya (`bestBy`). Greedy hanya melihat satu langkah ke depan, tanpa memikirkan balasan lawan.

10 Pengujian Win Rate dan Waktu Respons

tests/run.mjs : baris 148–199

```
148  // ---------------------------------------------------------------------------
149  console.log(`\n3. Strength (${GAMES} games per level per opponent, AI alternates first/second, 4 s
limit, Alpha-Beta on)`);
150  const timing = {};
151  const winRates = {};
152  {
153    console.log('   level   | depth | vs random | vs greedy');
154    for (const [level, depth] of Object.entries(DIFFICULTY)) {
155      timing[level] = [];
156      winRates[level] = {};
157      for (const [oppName, opp] of [['random', randomMove], ['greedy', greedyMove]]) {
158        let wins = 0;
159        for (let g = 0; g < GAMES; g++) {
160          const rng = mulberry32(BASE_SEED * 7 + depth * 100003 + g * 31 + (oppName === 'greedy' ? 17 :
0));
161          const aiSide = g % 2;
162          const s = createState();
163          let illegal = false;
164          while (winner(s) === null) {
165            let move;
166            if (s.turn === aiSide) {
167              const res = search(s, { depth, prune: true, timeLimitMs: AICFG.timeLimitMs, now:
performance.now.bind(performance) });
168              timing[level].push(res.timeMs);
169              move = res.move;
170            } else {
171              move = opp(s, rng);
172            }
173            if (!validateMove(s, move)) { illegal = true; break; }
174            applyMove(s, move);
175          }
176          check(`${level} vs ${oppName} game ${g}: only legal moves`, !illegal);
177          if (winner(s) === aiSide) wins++;
178        }
179        winRates[level][oppName] = (100 * wins) / GAMES;
180      }
181      console.log(`   ${level.padEnd(6)} | ${pad(depth, 5)} | ${pad(winRates[level].random.toFixed(0) +
'%', 9)} | ${pad(winRates[level].greedy.toFixed(0) + '%', 9)}`);
182      if (depth >= 2) check(`${level}: >= 90% vs random`, winRates[level].random >= 90,
`${winRates[level].random}%`);
183    }
184  }
185
186  // ---------------------------------------------------------------------------
187  console.log('\n4. Response time per AI move (Node, same code as the browser worker)');
188  {
189    console.log('   level   | moves | avg ms | p95 ms | max ms');
190    for (const [level, times] of Object.entries(timing)) {
191      const sorted = [...times].sort((a, b) => a - b);
192      const avg = times.reduce((a, b) => a + b, 0) / times.length;
193      const p95 = sorted[Math.floor(sorted.length * 0.95)];
194      const max = sorted.at(-1);
195      console.log(`   ${level.padEnd(6)} | ${pad(times.length, 5)} | ${pad(avg.toFixed(1), 6)} |
${pad(p95.toFixed(1), 6)} | ${pad(max.toFixed(1), 6)}`);
196      check(`${level}: max < 4000 ms`, max < 4000, `${max.toFixed(0)} ms`);
197      if (level !== 'expert') check(`${level}: max < 1000 ms`, max < 1000, `${max.toFixed(0)} ms`);
```

```
198     }
199 }
```

- **Setiap level** bermain 100 game vs acak dan 100 game vs greedy (`GAMES`; 20 kalau `--quick`).
- **Giliran pertama bergantian** (`aiSide = g % 2`) karena yang jalan pertama punya sedikit keuntungan; supaya adil, AI mendapat setengah game sebagai pemain pertama.
- **Seed per game** dibentuk dari level, nomor game, dan jenis lawan, jadi setiap game bisa diulang persis.
- **Setiap langkah dicek legal** dengan `validateMove`; kalau AI pernah membuat langkah ilegal, tes gagal.
- **Waktu respons**: `res.timeMs` dari setiap langkah AI disimpan, lalu dihitung rata-rata, p95 (diurutkan, ambil posisi 95%), dan maksimum.

11 Hasil Pengujian (Proposal Bab 2.2)

Semua angka di bawah berasal dari `node tests/run.mjs` (run penuh: 100 game per level per lawan). Hasil akhir run: **927 cek lulus, 0 gagal**. Jalankan sendiri sebelum presentasi supaya kamu tahu prosesnya: `node tests/run.mjs --quick` (±30 detik) atau tanpa `--quick` (±5 menit).

1. Kebenaran Alpha-Beta (Tabel 2.2.3, baris 1)

30 posisi uji dibuat dari permainan acak 4–8 giliran dengan seed `20261028 + k` (bisa diulang siapa pun). Alpha-Beta dan Minimax biasa memakai evaluasi, urutan langkah, dan tanpa batas waktu yang sama; bedanya hanya pemangkasan.

Depth	Skor identik	Langkah identik	Node Minimax	Node Alpha-Beta	Hemat	Cutoff	ms Minimax	ms Alpha-Beta
1	30/30	30/30	872	872	0,0%	0	10	6
2	30/30	30/30	24.540	7.050	71,3%	781	91	26
3	30/30	30/30	739.226	37.689	94,9%	2.242	2.063	191

Cara membaca: di depth 3, Alpha-Beta hanya memeriksa sekitar 1 dari 20 posisi yang diperiksa Minimax biasa, ±10× lebih cepat, dan tetap memilih skor dan langkah yang sama di 30 dari 30 posisi. Di depth 1 tidak ada penghematan karena belum ada simpul MIN yang bisa dipangkas (AI hanya melihat langkahnya sendiri).

2. Kekuatan AI / win rate (Tabel 2.2.3, baris 2)

Level	Depth	vs pemain acak	vs pemain greedy
Easy	1	100%	57%
Medium	2	100%	74%
Hard	3	100%	93%
Expert	4	100%	86%

Target proposal "Medium ke atas menang minimal 90% melawan pemain acak" **tercapai** (semua level 100%). Melawan greedy, win rate naik dari Easy ke Hard, tetapi **Expert (86%) sedikit di bawah Hard (93%)**. Jadi target "win rate naik seiring level" hanya tercapai sebagian. Jawaban jujur kalau ditanya:

- Dengan 100 game, selisih 7 poin persen masih dalam rentang acak (pemain greedy memilih acak di antara langkah yang sama bagusnya).
- Fungsi evaluasi BFS/DFS adalah perkiraan. Melihat lebih dalam dengan evaluasi yang tidak sempurna tidak selalu lebih baik (*horizon effect*).
- Pembatasan 12 kandidat jebakan bisa membuang langkah yang ternyata bagus di kedalaman lebih besar.

3. Waktu respons (Tabel 2.2.3, baris 3)

Level	Langkah diukur	Rata-rata	p95	Maksimum
Easy	931	0,1 ms	0,3 ms	0,4 ms
Medium	934	1,5 ms	4,4 ms	12,1 ms
Hard	914	18,5 ms	41,5 ms	82,7 ms
Expert	915	101,1 ms	316,3 ms	626,1 ms

Target "semua level di bawah 4 detik, Easy–Hard di bawah 1 detik" **tercapai jauh**. Angka ini diukur di Node.js dengan kode yang sama dengan Web Worker di browser; di laptop yang lebih lambat angkanya bisa lebih besar, tetapi masih jauh dari 4 detik. *p95* artinya 95% langkah selesai di bawah angka itu.

4. Hal yang belum dibuktikan oleh tes otomatis

- **Playtest** (±10 siswa, kuesioner skala 1–5) adalah uji manusia; hasilnya harus dicatat sendiri oleh tim. Jangan sebut angka playtest yang tidak benar-benar dilakukan.
- Waktu respons di browser bisa dicek langsung di panel **Otak AI** saat bermain (kolom Waktu).

tests/run.mjs : baris 16–22

```
16  function check(name, cond, detail = '') {
17    if (cond) passes++;
18    else {
19      failures++;
20      console.log(`  FAIL ${name}${detail ? ' :: ' + detail : ''}`);
21    }
22  }
```

12 Fitur Rally: Poin, Leaderboard, Admin

Fitur ini **tidak ada di proposal**; ditambahkan supaya game bisa dipakai di rally NPLC. Bukan bagian dari empat algoritma AI, tetapi mungkin ditanya saat demo.

Level	Pengali	Per langkah	Bonus menang
Easy	×1	+10	+200
Medium	×1,5	+15	+300
Hard	×2	+20	+400
Expert	×2,5	+25	+500

Hanya mode Single Player (lawan AI) yang memberi poin. Contoh: menang di Hard dengan 12 langkah = 12 × 20 + 400 = **640 poin**.

js/scoring.js : baris 64–96

```
64  export function scoreGame(moves, { mode = 'ai', level, humanSide, rules = SCORING }) {
65    const mult = rules.multiplier?.[level] ?? 0;
66    const out = {
67      valid: true, badMove: -1, finished: false, won: false, mult, steps: [],
68      humanMoves: 0, movePoints: 0, winPoints: 0, total: 0,
69    };
70    if (!pointsEnabled(mode) || !Object.hasOwn(DIFFICULTY, level) || !Array.isArray(moves)) return out;
71
72    const per = levelValues(rules, level);
73    const s = createState();
74    for (let i = 0; i < moves.length; i++) {
75      const m = moves[i];
76      if (!validateMove(s, m)) {
77        out.valid = false;
78        out.badMove = i;
79        return out;
80      }
81      const side = s.turn;
82      applyJump(s, m.jump);
83      applyTrap(s, m.trap);
84      if (side === humanSide) out.humanMoves++;
85      out.steps.push(side === humanSide ? per.move : 0);
86      endTurn(s);
87    }
88
89    const w = winner(s);
90    out.finished = w !== null;
91    out.won = w === humanSide;
92    out.movePoints = out.humanMoves * per.move;
93    if (out.won) out.winPoints = per.win;
94    out.total = out.movePoints + out.winPoints;
95    return out;
96  }
```

- **Anti curang:** browser hanya mengirim daftar langkah. Server (`api/score.js`) memutar ulang semua langkah dengan `rules.js` yang sama, menolak langkah ilegal atau game yang belum selesai, lalu menghitung poin sendiri.

- **Leaderboard:** total poin per tim dari semua game, urut total → jumlah menang → yang lebih dulu mengirim. Disimpan di Upstash Redis lewat Vercel.
- **Admin:** `/admin.html` dengan kunci `ADMIN_KEY` bisa mengubah poin per langkah, bonus menang, dan pengali tiap level.

13 Latihan

Latihan 1: BFS dengan tangan

Knight di b1 (papan kosong). Sebutkan semua kotak berjarak 1. Berapa jarak ke b2?

Jawaban: jarak 1: a3, c3, d2. b2 butuh 3 lompatan (contoh: b1 → d2 → c4 → b2). Kotak yang bersebelahan langsung justru jauh untuk knight.

Latihan 2: Aturan seri

Sebuah kotak berjarak 2 dari P1 dan 2 dari P2, dan sekarang giliran P2. Kotak itu masuk wilayah siapa? Mengapa?

Jawaban: P2, karena P2 yang jalan berikutnya dan akan mencapainya lebih dulu (`terr += turn === 0 ? 1 : -1`).

Latihan 3: Coba di kode

1. Di `evaluate()`, ubah bobot mobilitas 0,5 menjadi 0, jalankan `node tests/run.mjs --quick`, bandingkan win rate vs greedy. (Kembalikan setelahnya.)
2. Di `css/style.css`, ubah `--cell-a` dan `--cell-b` lalu buka game. Semua kotak ikut berubah karena memakai variabel.

14 Pertanyaan Presentasi yang Mungkin Muncul

T: Kenapa BFS, bukan DFS, untuk menghitung jarak?

J: BFS menelusuri per lapisan, jadi saat sebuah kotak pertama kali ditemukan, jaraknya pasti terpendek. DFS tidak menjamin jarak terpendek.

T: Apa itu wilayah dan kenapa dipakai di evaluasi?

J: Kotak yang bisa dicapai knight kita lebih cepat dari knight lawan. Karena kalah = kehabisan lompatan, punya lebih banyak kotak "milik sendiri" berarti bisa bertahan lebih lama.

T: Kenapa ada 0,5 × mobilitas?

J: Sebagai pemecah seri. Bobotnya kecil supaya wilayah tetap faktor utama, tetapi posisi dengan lompatan lebih banyak sedikit diutamakan.

T: Bagaimana win rate diukur supaya adil?

J: 100 game per level per lawan, AI bergantian jalan pertama, seed tercatat, batas waktu 4 detik aktif, dan setiap langkah dicek legal.

T: Kenapa animasi tidak membuat AI lambat?

J: AI berjalan di Web Worker (thread lain). Animasi di thread utama, jadi keduanya tidak saling menunggu.

T: Bagaimana pemain tahu lompatan mana yang legal?

J: `game.view()` mengisi `legal` dengan `legalJumps()`, lalu `render()` memberi class `legal` yang menampilkan titik di kotak itu. Di fase jebakan, kotak tujuan lawan diberi class `threat` (bergaris).

15 Checklist Sebelum Presentasi

- ☐ Bisa menghitung BFS knight dengan tangan di papan kecil.
- ☐ Bisa menjelaskan papan wilayah posisi awal dan aturan seri.
- ☐ Bisa menunjuk `render()` dan menjelaskan class CSS yang dinyalakan.
- ☐ Hafal hasil win rate (100% vs acak; 57/74/93/86% vs greedy) dan waktu (Expert maks. 626 ms).
- ☐ Bisa menjelaskan kenapa Expert vs greedy sedikit di bawah Hard.
- ☐ Bisa menjelaskan alur satu giliran (Bab 4) tanpa melihat catatan.